Laborator 2

2. Primitive, collectii

2.1 Primitive (tipuri de date de baza): int, long, float, double, char, byte

2.1.1 Numere intregi: int

```
public static void main(String[] args) {
    int i = 0;
    int j =  i + 10;
    System.out.println("i = " + i);
    System.out.println("j = " + j);
}
```

2.1.2 Numere intregi: long

```
public static void main(String[] args) {
    long i = 0;
    long j =  i + 10;
    System.out.println("i = " + i);
    System.out.println("j = " + j);
}
```

2.1.3 Numere cu virgula mobila: double / float

```
public static void main(String[] args) {
    float f = 0.5f;
    double i = 0.01;
    double j =  f + 10;
    System.out.println("i = " + i);
    System.out.println("j = " + j);
}
```

2.1.4 Conversie String <-> int, long, double

```
public static void main(String[] args) {
    String s1 = "12";
    int v1 = Integer.parseInt(s1);
    System.out.println("v1 = "+v1+"  binar = "+Integer.toBinaryString(v1));

    String s2 = Long.toString(1000L + (long)Integer.MAX_VALUE);
    System.out.println("s2 = "+s2);
    long v2 = Long.parseLong(s2);
    System.out.println("v2 = "+v2);

    String s3 = "12.34";
    double v3 = Double.parseDouble(s3);
    double v4 = Double.valueOf(s3);
    System.out.println("v3 = "+v3);
    System.out.println("v4 = "+v4);
}
```

2.2 Interfata java.util.Collection

Operatii:

```
Collection collection = new ArrayList(); //crearea colectiei
collection.add(new Object()); //adaugarea unui obiect
collection.add(new Object());//adaugarea altui obiect

boolean wasElementRemoved = collection.remove(anObject); //eliminarea //unui
                                    element din colectie

Collection collection2 = new LinkedList();
collection2.add(new Object());
collection2.addAll(collection); //adaugarea unei colectii la alta colectie
collection2.removeAll(collection); //eliminarea elementelor unei colectii din
                                    alta colectie

//verificarea existentei unui element in lista
boolean containsElement = collection2.contains("an element");
boolean containsAll     = collection2.containsAll(elements); //verificare
                                    existentei mai multor elemente

Iterator iterator = collection.iterator();
while(iterator.hasNext()){          //parcurgerea colectiei cu iterator
  Object object = iterator.next();
  System.out.println(object);
}

for(Object object : collection) {   //parcurgerea colectiei cu for-each
    System.out.println(object);
}
```

Interfete care mostenesc interfata java Collection:

- List
- Set
- SortedSet
- NavigableSet
- Queue
- Deque

Mai multe la http://tutorials.jenkov.com/java-collections/collection.html

2.3 Interfata java.util.List

2.3.1 Implementari ale interfetei java.util.List:

- java.util.ArrayList
- java.util.LinkedList
- java.util.Vector
- java.util.Stack

```
List listA = new ArrayList();
List listB = new LinkedList();
List listC = new Vector();
List listD = new Stack();
```

2.3.2 Operatii cu liste:

```
List<String> list1 = new ArrayList<String>();
list1.add("elem a");  //adaugare element
list1.add(1, "elem B");  //adaugare element la o pozitie data
list1.add(2, "elem B");

List list2 = new ArrayList<String>();
list2.addAll(list1);

String elementA = list1.get(0);  //obtinerea unui element din lista de pe
                                 //pozitia 0
String elementB = list1.get(1);  //obtinerea unui element din lista de pe
                                 // pozitia 1

int index1 = list.indexOf(elementA);
int index2 = list.indexOf(elementB);

int lastIndex = list.lastIndexOf(elementB);//obtinerea ultimei aparitii unui
                                           //element


boolean containsElement = list.contains("elem a"); //verificarea existentei
                                                   // unui element

list.remove(elementA);                             //eliminarea unui element
                                                   // din lista
list.remove(1); //eliminarea unui element de pe o anumita pozitie
list.clear(); //eliminarea tuturor elementelor din lista.
int size = list.size(); //dimensiunea listei. Nr de elemente din lista.
list.retainAll(otherList); //retine in lista doar elementele prezente si in
                           //otherList

List sublist = list.subList(1, 3); //crearea unei subliste cu elemente din
                                   //prima lista.

Object[] objects = list.toArray(); //convertirea listei intr-un array
String[] objects1 = list1.toArray();
```

Laborator 2

```
String[] values = new String[]{ "one", "two", "three" };
List list = Arrays.asList(values);  //covertirea unui array la List

Iterator<String> iterator = list.iterator();
while(iterator.hasNext()){        //parcurgere lista cu iterator
    String obj = iterator.next();
}

for(Object element : list) {      //parcurgere lista cu for-each
    System.out.println(element);
}

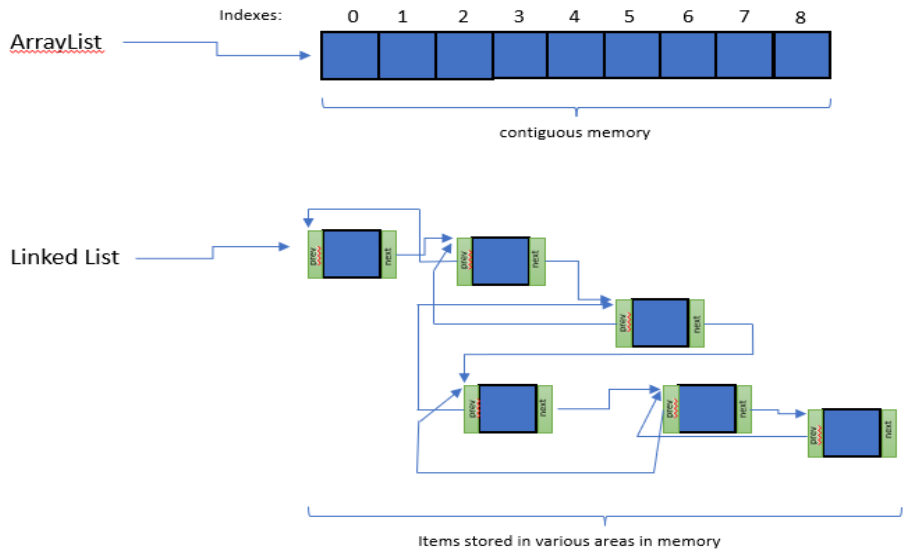
for(int i=0; i < list.size(); i++) {  //parcurgere lista cu for i to n
    Object element = list.get(i);
}

Stream<String> stream = stringList.stream();
stream.forEach( element -> { System.out.println(element); }); //parcurgere
                                    // lista folosind stream api (java 1.8+)

Collections.sort(list);  //sortarea unei liste

//sortare cu comparator specific
List<Car> list = new ArrayList<>();
list.add(new Car("Volvo V40" , "XYZ 201845", 5));
list.add(new Car("Citroen C1", "ABC 164521", 4));
list.add(new Car("Dodge Ram" , "KLM 845990", 2));
Comparator<Car> carBrandComparator = new Comparator<Car>() {
    @Override
    public int compare(Car car1, Car car2) {
        return car1.brand.compareTo(car2.brand);
    }
};
Collections.sort(list, carBrandComparator);
```

2.3.3 ArrayList vs LinkedList



Comparatie a performantei (evaluare grosiera caci depinde de implementarea jvm-ului, a hardware-ului, a politicii garbage collector):

Operation	ArrayList (10^6 elements)	LinkedList (10^6 elements)
add(e) (end)	~10 ms	~12 ms
add(0, e)	~500 ms	~1 ms
get(i)	~1 ms	~300 ms
remove(i)	~200 ms	~300 ms

	ArrayList	LinkedList
get()	O(1)	O(n)
add()	O(1)	O(1) amortized
remove()	O(n)	O(n)

2.4 Set

Un Set este similar cu List cu exceptia faptului ca in Set acelasi element nu poate fi gasit de mai multe ori, pe cand intr-o lista acelasi element poate fi gasit pe mai multe pozitii.

Implementari:

- java.util.EnumSet
- java.util.HashSet
- java.util.LinkedHashSet
- java.util.TreeSet

```
Set setA = new EnumSet();
Set setB = new HashSet();
Set setC = new LinkedHashSet();
Set setD = new TreeSet();
```

Operatii pe Set:

```
Set setA = new HashSet();
setA.add("element 1");   //adaugare elemente in Set
setA.add("element 2");
set.remove("object-to-remove"); //eliminare element din Set
set.clear();                    //eliminare tuturor elementor din Set

Set set2 = new HashSet();
set2.addAll(set);               //adaugare elemente din alt Set
set.removeAll(set2);            //eliminare elemente din alt Set

int size = set.size();           //nr de elemente in Set
boolean isEmpty = set.isEmpty();     //verificare daca setul este gol
boolean isEmpty = (set.size() == 0); //verificare daca setul este gol
boolean containsA = set.contains("Some");//verificare daca setul contine un
                                         // element

//Generics
Set<MyObject> set = new HashSet<MyObject>(); //folosire 'generics'
```

Laborator 2

```
// conversie Set in List
List list = new ArrayList();
list.addAll(set);


Iterator iterator = set.iterator();
while(iterator.hasNext(){  //parcurgere Set cu iterator
  String element = (String) iterator.next();
}

for(Object object : set) {  //parcurgere Set cu for-each
    String element = (String) object;
}

Stream stream = set.stream();//parcurgere cu stream si lambda (java 1.8+)
stream.forEach((element) -> { System.out.println(element); });
```

Stocarea datelor in HashSet se face pe baza valorii hash (rezultatul functiei hashCode()) iar aceasta poate schimba ordinea in care vor fi stocate datele in memorie. De exemplu codul urmator:

```
HashSet<String> hset = new  HashSet<String>();
String var1 = "TEST1";
String var2 = "TEST2";

hset.add(var1);
hset.add(var2);

System.out.println("Set contents: ");
hset.forEach(item -> System.out.println(item));
System.out.println("var1 hashcode = "+ var1.hashCode());
System.out.println("var2 hashcode = "+ var2.hashCode());
```

Produce rezultatul:

```
Set contents:
TEST2
TEST1
var1 hashcode = 79713759
var2 hashcode = 79713760
```

Note:

1. Intr-un set ordinea interna **nu** este garantata.
Intr-o lista ordinea interna se pastreaza si elementele pot fi iterate in acea ordine.

2. Cautarea in Set se face cu O(1). Metoda contains() pentru Liste se va executa in O(n) iar pentru Set-uri in O(1).

2.5 Probleme propuse

2.5.1 Fie declaratiile:

```
List<Integer> x = new ArrayList();
List<Integer> y = new ArrayList();
List<Integer> xPlusY = new ArrayList(); //a
Set<Integer>  zSet = new TreeSet();     //b
List<Integer> xMinusY = new ArrayList();//c
int p = 4;
List<Integer> xPlusYLimitedByP = new ArrayList(); //d
```

Dati valori aleatorii elementele listelor x şi y in domeniul [0..10] şi apoi ordonati aceste doua liste crescător.
Numarul de elemente pt lista x este 5. Numarul de elemente pt lista y este 7.
Să se scrie cod pentru a obţine colectii cu elemente ordonate crescător, după efectuarea următoarelor operaţii:
a) xPlusY conţine toate elementele şirurilor x şi y;
b) zSet conţine numai valorile comune din ambele şiruri, luate o singură dată;
c) xMinusY conţine valorile din şirul x care nu se află în şirul y;
d) xPlusYLimitedByP este mulţimea elementelor din x şi y ce nu depăşesc valoarea p.

Creati un nou packet (ex. lab2) in aplicatia lab1 de data trecuta, si adaugati sursele in acest pachet, apoi faceti commit si push.

2.5.2 Deschideti proiectul **Students** de data trecuta si:
a) Să creeze o listă din lista de instante Student deja definte. Afisati continutul acestei liste folosind for-each
b) Să se afiseze dacă in lista este prezent studentul

```
new Student(120, "Alis",  "Popa", "TI21/2");
```

c) Să se afiseze dacă in lista este prezent studentul

```
new Student(112, "Maria",  "Popa", "TI21/1");
```

Pentru b) si c) implementati o functie care returneaza true/false daca un student exista in lista cu același prenume, nume, formațieDeStudiu
Creati un nou commit si push cu aceste noi modificari.

2.5.3 Tema de casa

Rescrieti functia pentru b) si c) astfel incat cautarea sa se realizeze cu O(1).

Creati un nou commit si push cu aceste noi modificari. Mesajul commit-ului sa fie „Tema casa laborator 2”.

2.6 Referinte

Java code conventions:
https://www.oracle.com/java/technologies/javase/codeconventions-contents.html

Lïst
http://tutorials.jenkov.com/java-collections/list.html

Set
http://tutorials.jenkov.com/java-collections/set.html

Java Collections
http://tutorials.jenkov.com/java-collections/index.html

Comparator Interface in Java with Examples
https://www.geeksforgeeks.org/comparator-interface-java/

Hashing
https://www.geeksforgeeks.org/hashing-in-java/

ArrayList vs LinkedList
https://www.baeldung.com/java-arraylist-linkedlist